

ExpAnz — Automated Bank Statement Analyzer

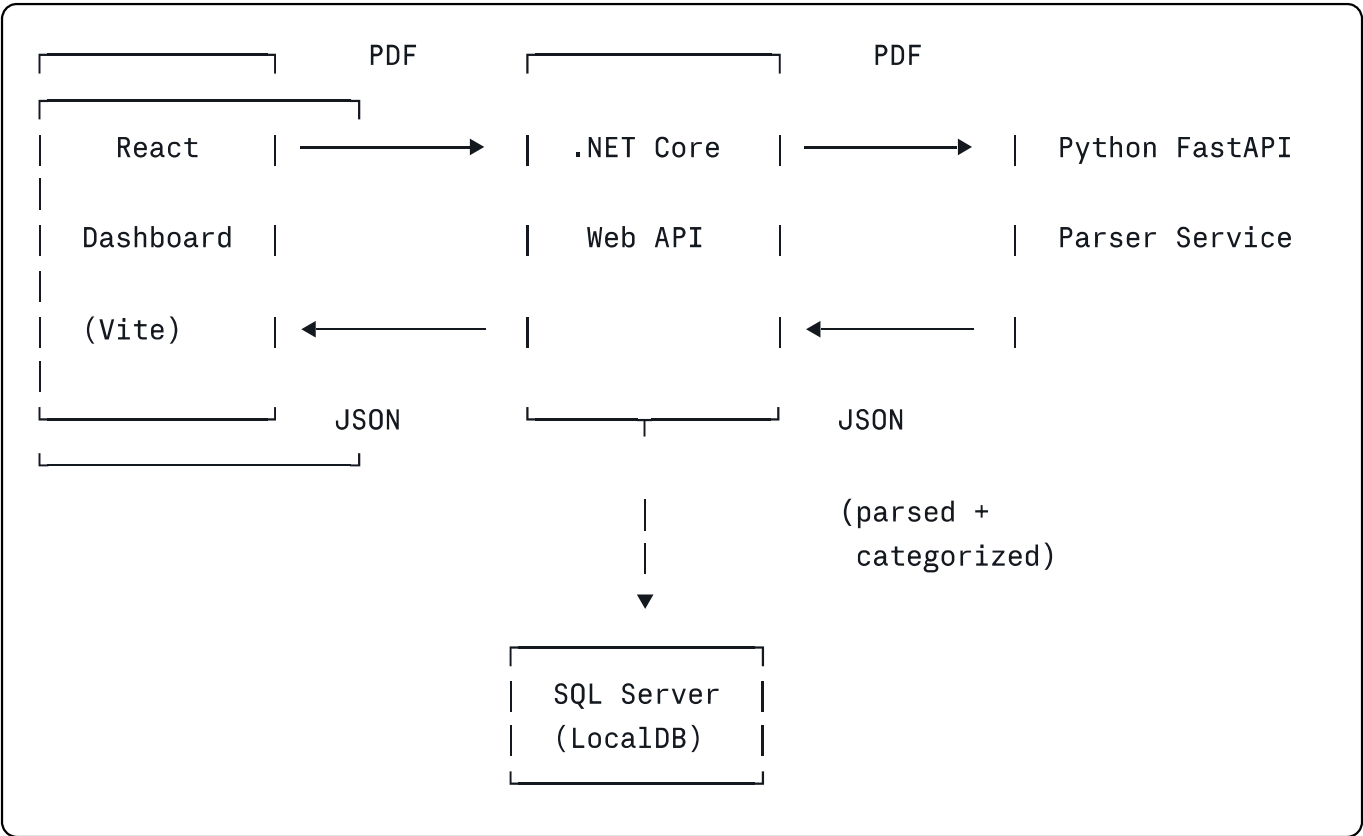
A full-stack personal finance tool that turns raw bank statement PDFs into an interactive spending dashboard — with zero manual data entry.

1. The Problem

Traditional budgeting apps require you to enter every transaction by hand. That friction is why most people abandon them. ExpAnz removes it entirely: you drag in a bank statement PDF, and the system extracts, categorizes, deduplicates, and visualizes every transaction automatically.

2. Architecture

Three independent services, each doing one job, communicating over HTTP.



Layer	Technology	Responsibility
Frontend	React + Vite, Recharts	Dashboard, charts, drill-down, upload UI
Backend	.NET Core Web API, EF Core	Persistence, aggregation, deduplication, orchestration
Parsing	Python, pdfplumber, FastAPI	PDF extraction, categorization, reconciliation
Database	SQL Server (LocalDB)	Transaction and category storage

Why three services rather than one? Each language is used where it's strongest — Python for document parsing, .NET for a typed API and ORM, React for interactive UI. The clean HTTP boundaries mean any one can be replaced or scaled independently.

3. The PDF Parser (the hard part)

Bank statements are deceptively difficult to parse. HSBC's layout has no gridlines and conveys its table structure purely through spatial alignment.

The approach: coordinate-based extraction. Rather than reading flat text (which scrambles the columns), the parser uses `pdfplumber` to get the exact x/y position of every word on the page. Column membership is determined by horizontal position; row membership by vertical position.

The structural challenges solved:

- **A transaction spans two visual lines.** The payee sits on line one; the description tail and the amount sit on line two. The parser assembles transactions incrementally — opening a "pending" transaction on description text, absorbing words across lines, and finalizing when the amount is found.
- **Dates are sparse.** A date appears once and every transaction beneath it inherits it. The parser carries the last-seen date forward.
- **Multiple transactions stack under one date.** One date does not equal one row.
- **Direction is positional.** Whether an amount is money-out or money-in is determined by which column it falls in (x-position), not by any sign or marker.
- **Page headers and balance markers** ("BALANCE BROUGHT FORWARD") must be detected and skipped, including where header text bleeds into descriptions at page boundaries.

Reconciliation — the correctness guarantee

Every HSBC statement prints its own Account Summary: opening balance, total payments

in, total payments out. After parsing, the system sums its extracted transactions and asserts they match those stated totals **to the penny**.

This is not a nice-to-have. During development it caught a silent regression where incoming payments without a transaction-type code were being dropped — a bug that would have quietly under-reported income with no visible error. Any parser that fails reconciliation is a parser that is lying to you.

4. Categorization

A two-level scheme: a broad **category** plus a clean canonical **merchant** (acting as a subcategory), enabling drill-down from "Shopping — £90" into "Primark £21, Amazon £16, TK Maxx £6".

Ordered keyword rules, first match wins. Specificity beats generality, so rule order is load-bearing:

```
python
(["longdan ltd"], "Income"),      # the employer — must come first
(["longdan", "lidl", ...], "Groceries"), # the grocery shop
```

Reverse those two lines and your salary is categorized as a grocery run.

Merchant normalization maps messy raw text (`AMZNMktplace*NN9B5 amazon.co.uk`) to a clean name (`Amazon`).

Why rules and not machine learning? The set of banks and merchants is small and known. Rules are more accurate here, fully debuggable, and correctable in seconds. ML would add opacity for no accuracy gain. It becomes worth considering only once there is a large corpus of user-corrected labels to train on.

5. Deduplication

Uploading overlapping statements (March–April, then April–May) would otherwise double-count the shared days.

Each transaction gets a **SHA-256 fingerprint** of `(date | description | amount)`. Before insert, the API checks incoming fingerprints against existing ones and skips matches — deduplicating both against the database and within the incoming batch.

Uploads are therefore **idempotent**: re-uploading the same statement inserts nothing.

parsed: 53, inserted: 16, skippedDuplicates: 37

Known limitation: two genuinely distinct transactions with identical date, description, and amount (e.g. two £1.75 TfL taps on the same day) will be treated as duplicates. This is a fundamental limit of PDF data, which carries no unique transaction ID. A bank API would solve it; a PDF cannot.

6. Data Model

Transaction

Field	Type	Notes
Id	int	Primary key
Date	DateTime	
Description	string	Raw text from the statement
Amount	decimal(18,2)	Negative = money out, positive = money in
Category	string	Broad bucket
Merchant	string	Clean name (subcategory)
SourceBank	string	HSBC / Monzo
StatementMonth	string	e.g. "2026-04"
Fingerprint	string	SHA-256 dedup hash

Category — Id, Name, Color. Seeded, and manageable at runtime via the API.

Money is `decimal`, **never** `float`. Floating-point arithmetic drifts, and drift in a financial ledger is unacceptable. EF Core is explicitly configured with `HasPrecision(18, 2)`.

7. API Surface

Endpoint	Purpose
<code>GET /api/transactions</code>	All transactions

Endpoint	Purpose
<code>POST /api/transactions</code>	Create one
<code>POST /api/transactions/bulk</code>	Batch insert with dedup
<code>PUT /api/transactions/{id}/category</code>	Reassign a transaction's category
<code>GET /api/summary?startDate=&endDate=</code>	Spending by category over a date range
<code>GET /api/summary/monthly?startDate=&endDate=</code>	Spending by category per month (trends)
<code>GET /api/categories</code>	List categories
<code>POST /api/categories</code>	Create a category
<code>DELETE /api/categories/{id}</code>	Delete (reassigns orphans to Uncategorized)
<code>POST /api/upload</code>	Upload a PDF → parse → dedup → save
<code>POST /parse</code> (<i>Python service</i>)	PDF in, categorized transactions + reconciliation totals out

The out / in / net distinction

Summary endpoints return three figures per category rather than one:

- `totalOut` — money spent (sum of negatives)
- `totalIn` — money received back (refunds, returns, repayments)
- `net` — the difference

This matters. Netting money-in against money-out produces a figure that is neither "what I spent" nor "what it cost me". A category where £600 goes out and £300 comes back is not the same as a category where £300 goes out — but a single net figure makes them look identical. **The dashboard charts** `totalOut`. A chart labelled "Spending by category" shows spending.

Likewise, **income means the Income category only**. Positive amounts elsewhere (a refund, a repayment) are money returning, not money earned, and are never counted as income.

8. Dashboard Features

- **Drag-and-drop PDF upload** with live parse feedback and automatic data refresh

- **Doughnut chart** of spending by category, with fixed per-category colours (never reassigned by rank, so a category's colour is stable across filters)
 - **Drill-down** — click a category to see its merchant breakdown; click a merchant to filter further
 - **Date-range filtering**
 - **Category management** — add, delete, and reassign transactions between categories, persisted to the database
 - **Uncategorized review** — a filtered view surfacing transactions the rules didn't match
 - **Exclude internal transfers** toggle — separates true consumption from money moving between your own accounts
 - **Month-over-month comparison** — trend chart, comparison table with % change, automatic "biggest mover" callouts, and partial-month flagging (statements don't align to calendar months, so incomplete edge months are marked rather than silently misread as a collapse in spending)
-

9. Running the Project

Three services:

```
bash

# 1. .NET API → https://localhost:7131
#   (run from Visual Studio, or `dotnet run`)

# 2. Python parser service → http://localhost:8000
cd budget-parser
venv\Scripts\activate
uvicorn parse_service:app --reload --port 8000

# 3. React dashboard → http://localhost:5173
cd budget-analyzer-web
npm install
npm run dev
```

The Python service is only required for uploading statements; browsing the dashboard needs only the API and frontend.

10. Engineering Principles Applied

Reconciliation as ground truth. The parser is only trusted because it proves itself against the statement's own printed totals on every run. Correctness in financial software must be *verifiable*, not merely *plausible*.

Test bottom-up. Each layer was verified in isolation — endpoints in Postman before any UI, the Python service standalone before .NET called it — so bugs surfaced where they were introduced.

Avoid premature abstraction. The scope was deliberately narrowed to one bank first. Rules were chosen over ML. A bulk endpoint was built because the pipeline genuinely needed batch inserts, not because it looked sophisticated.

Real data reveals real gaps. Test data is agreeable and tells you nothing. Every meaningful bug — the dropped incoming payments, the two-line description merge, the page-header bleed, the netting that flattered the totals — was found by running actual statements through the system.

Don't let the presentation flatter the numbers. The most important design decisions in this project were about what *not* to hide: charting gross spending rather than net, keeping income honest, and refusing to let money-in silently cancel money-out. A tool that tells you what you want to hear is worse than no tool at all.

11. Roadmap

- **Monzo parser** — a second bank, feeding the same categorization and dedup pipeline
- **Rule learning** — recategorize a merchant once in the UI, and have it persist for future statements
- **Budgets** — per-category monthly limits with progress tracking
- **AI insights** — narrative summaries over *aggregated* data (the code computes, the model narrates — never the reverse)